A.U.R.A. — Full Presentation Script (17 Slides)

How to use this: Read each slide's script in your own words — don't memorize it word for word, or it will sound robotic. The wording here is chosen carefully so you don't accidentally overclaim or say something a panel member can trap you on. A few ground rules are listed below before the script.

Ground Rules (Read This First)

1. **Never say "100% secure" or "unhackable."** Always say things like *"this reduces the risk"* or *"this adds a layer of protection."* No security system is ever 100% guaranteed, and a panel member will immediately push back if you claim that.
2. **Don't claim finished, tested results you don't have.** Say *"the system is designed to..."* or *"our approach is..."* rather than *"we tested this and it works perfectly."* This is a proposal/architecture-stage project, so speak about it as a design that is backed by research — not as a finished product with benchmark numbers.
3. **On the WhatsApp/LinkedIn automation part**, never say "we bypass detection" or "we get around their rules." Say *"we reduce the risk of automated-behavior detection during controlled testing"* and, if pushed, openly say you are aware this is a policy consideration and you will keep testing limited and controlled. Don't argue that it's fully allowed — just be honest that it's a known constraint.
4. **Don't say your project "replaces" or is "better than everything"** like ChatGPT/Claude/Copilot. Say it solves a *different, specific* problem: local privacy, zero ongoing cost, and hardware-enforced safety — not that it's smarter or more capable overall.
5. **If you don't know the answer to a question, don't guess or make up a number.** Say: *"That's something we plan to measure during implementation/testing — right now this is at the design and architecture stage."* This is a safe, honest answer that panels respect.

Slide 1 — Title Slide (A.U.R.A.)

What to say:

"Good [morning/afternoon] everyone. Our project is called A.U.R.A. — Air-gapped Unified Responsive Agent. In simple words, it is a locally-run, multi-agent system that automates tasks on your computer — without needing the internet or any cloud AI service. Our team is [say

names], and today we'll walk you through the problem we are solving, how our system works, and the technology behind it."

Slide 2 — Project Domains & Nature

What to say:

"Before going into the project itself, we want to show that this project sits across four core computer science domains. First, Artificial Intelligence — because we use local language models for reasoning and multi-agent coordination. Second, Computer Automation — because the system interacts directly with the operating system to complete tasks. Third, Cybersecurity — because privacy, access control, and agent safety are central to our design. And fourth, Edge Computing — because everything runs on local hardware, including a Raspberry Pi companion device, instead of relying on cloud servers. So this project isn't narrow — it combines four active research areas into one system."

Slide 3 — Technical Problem Statement

What to say:

"Now let's talk about the actual problem. Most existing AI assistants today have five limitations. First, Cloud Dependency — they send your commands and often your private context to remote servers. Second, Limited OS Control — they can chat with you, but they struggle to reliably perform complex, multi-step tasks directly on your computer. Third, Weak Context Persistence — they tend to forget your preferences and file locations once a session ends. Fourth, Security Risks — because these agents read information from the web, they can be exposed to prompt injection attacks, where hidden malicious instructions try to make the AI do something harmful. And fifth, there's usually No Safe Human Approval Layer — a simple 'Are you sure?' pop-up is often the only thing standing between the AI and a risky action. This diagram shows it clearly: once your command and your private context cross into a remote cloud API, that trust boundary is crossed, and you no longer fully control what happens to your data."

Slide 4 — A.U.R.A. — Proposed Solution

What to say:

"So how do we respond to these problems? A.U.R.A. is built around six design choices. One, Local-First Intelligence — reasoning happens using quantized language models running on your own device, not in the cloud. Two, a Multi-Agent Architecture — a Brain, a Manager, and specialized Sub-Agents divide complex tasks into smaller, controlled operations. Three, Direct OS Interaction — we use accessibility APIs to target specific application elements directly, instead of relying only on simulated mouse clicks. Four, Vision Fallback — if accessibility information isn't available, a local vision model can still locate what it needs on screen. Five, Security by Design — untrusted web content is sanitized before it ever reaches the AI, and agents only operate with restricted permissions. And six, Human-in-the-Loop Protection — high-risk actions require physical biometric approval before they execute. In short: local intelligence, plus controlled automation, plus security, plus human oversight — all in one system."

Slide 5 — Why Local Computer Automation Matters

What to say:

"Stepping back to the real-world side: the modern computer user faces three connected problems. First, too many repetitive tasks — opening applications, managing files, searching for information, switching between windows. Second, sensitive data — personal documents, credentials, work files, private conversations, and browsing activity, all of which most people don't want leaving their machine. And third, trust and control — people want automation, but they don't want to hand an AI unrestricted access to their entire system. So the core challenge we're answering is this: can an AI agent perform useful computer tasks while keeping the user's data, reasoning, and execution locally controlled? That question is really what A.U.R.A. is designed around."

Slide 6 — Conventional Assistant vs. A.U.R.A.

What to say:

"This table compares a typical AI assistant with our design. Conventional assistants usually rely on cloud-based processing and API-dependent intelligence — A.U.R.A. uses local processing with local quantized models. Conventional assistants are mostly chat-oriented with a single-assistant workflow — we use task-oriented OS automation with multiple coordinated agents. Where conventional tools often have limited persistent context, we use hierarchical long-term memory. Where accessibility is often a secondary feature and vision is the primary interaction method, we flip that — accessibility-first, with vision only as a fallback. And on the security side, where broad permissions and a basic confirmation dialog are common, we use restricted-permission execution plus biometric approval for high-risk actions, along with

encrypted local storage. Our design goal, in one line, is: useful automation without sacrificing local privacy and user control."

Slide 7 — How A.U.R.A. Works

What to say:

"Here's the high-level flow of how a task moves through the system, in six steps. Step one, User Command — a natural-language instruction comes in, either typed or spoken. Step two, the Brain interprets the user's intent and identifies the overall task. Step three, the Manager breaks that task into smaller operations and decides which agents are needed. Step four, specialized Sub-Agents handle their specific domains — we have a Browser Agent for web interaction, a File/OS Agent for locating and manipulating files, a Communication Agent for messaging tasks, a Research Agent for gathering information, and a Security Agent that validates permissions. Step five is Local Execution — using controlled OS interfaces and approved tools. And step six is Verification — the result is checked, the state is recorded, and the outcome is reported back to the user. Importantly, shared context travels with the task through every stage, so all the agents stay aligned on the same goal."

Slide 8 — A.U.R.A. Core Capabilities (Features 1–5)

What to say:

"Let's go through our first five core features. One, Multi-Agent Orchestration — the Brain interprets the command, the Manager decomposes the task, and specialized agents perform the domain-specific work, all while sharing context. Two, Hybrid Local Reasoning — this is our two-tier model design. A lightweight BitNet b1.58 2-billion-parameter model handles fast routing and simple decisions, while a larger quantized 7-to-8-billion-parameter model handles more complex reasoning — both running fully offline. Three, Background OS Automation — we use accessibility APIs to identify and interact directly with UI elements, so tasks can run without repeatedly taking over the user's foreground window. Four, Vision Fallback — when accessibility metadata can't locate something, a local vision model steps in using screenshot-based understanding, which helps the system adapt when an application's layout changes. And five, Long-Term Memory — the system stores useful context such as habits, preferences, and file references, so it doesn't lose continuity between sessions."

Slide 9 — Security, Control & Hardware (Features 6–10)

What to say:

"Now the next five features, which are more about security and hardware. Six, Zero-Trust Security — untrusted web content is sanitized before it reaches the AI, and restricted execution permissions limit the impact if a task is ever compromised. Seven, Controlled Interaction — we apply platform-aware rate limits and interaction policies, which are designed to reduce excessive or unsafe automated activity. Eight, Biometric Human-in-the-Loop Security — for high-risk actions, execution is frozen and paused until fingerprint verification is approved. Nine, the Raspberry Pi Companion — this is our edge hardware node. It processes locally, runs A.U.R.A.'s reasoning and agent logic, and gives a verified response back — supported by a wake-word microphone, a fingerprint sensor, and a small projector for feedback. And ten, Encrypted Local Storage — our local state and memory databases, ChromaDB and SQLite, are protected using AES-256 encryption, and the keys never leave the device."

Slide 10 — Multi-Agent Task Execution (Worked Example)

What to say:

"To make this concrete, here's a worked example: 'Find the project report, summarize it, and save the summary in the Documents folder.' This goes through six steps. Step one, User Input — the natural-language command is received locally by the interface; no content is sent to a cloud API. Step two, Intent Understanding — the Brain Agent identifies the user's goal, the applications involved, the information required, and the expected final result. Step three, Task Decomposition — the Manager Agent breaks this into concrete steps: locate the report, read the relevant content, generate a summary, open the target folder, save the output, and verify completion. Step four, Agent Routing — the task is dispatched to the right domain agents; here, mainly the File/OS agent. Step five, Controlled Execution — the system checks whether the accessibility API can find the needed UI element; if yes, it interacts directly, and if not, it falls back to vision-based understanding. And step six, Result Verification — the action is confirmed, memory is updated where appropriate, the execution state is recorded, and the result is returned to the user."

Slide 11 — Hybrid Quantized AI Architecture

What to say:

"This slide explains why we use two models instead of one. Tier one is BitNet b1.58 2B — designed for fast classification, intent routing, simple decisions, and low-resource inference. Tier two is a quantized 7-to-8-billion-parameter model — designed for multi-step reasoning, task planning, context interpretation, and more complex decision-making. Every incoming operation

goes through a complexity classification step: if it's simple, BitNet b1.58 handles it; if it's complex, it's routed to the larger quantized model. Both run through llama.cpp for local CPU inference — there is no outbound API call at any stage. The reason we chose this hybrid approach instead of one large model is practical: it reduces computational overhead, makes better use of CPU resources, lowers the local inference cost, and makes offline operation more realistic on typical laptop hardware without a dedicated GPU."

Slide 12 — Memory, Security & Agent Control

What to say:

"This slide goes a bit deeper into four internal modules. Module one, Long-Term Memory — a hierarchical memory layer storing session context, workflow patterns, and durable preferences, retrieved locally through ChromaDB. Module two, the Security Layer — this follows a zero-trust execution model: it sanitizes untrusted web content, restricts agent permissions, keeps untrusted input separate from trusted instructions, and limits unnecessary OS access, which reduces the surface for indirect prompt injection. Module three, Biometric Control — this is our high-risk action guard: for things like file deletion or sensitive credential access, the flow is request, then risk check, then the action is frozen, and only proceeds once fingerprint approval is given. And module four, Encrypted State — our memory database, ChromaDB, and our state database, SQLite, are both AES-256 encrypted at rest, and managed entirely on-device."

Slide 13 — A.U.R.A. End-to-End Process (Flowchart)

What to say:

"This flowchart shows the complete journey of a single command from start to finish. It begins with the user command in natural language or voice. The system checks if the input is understandable — if not, it asks for clarification rather than guessing. If it is understood, the Brain Agent interprets intent, and the Manager Agent decomposes the task and selects the right specialized agent. Before execution, there's a permission and risk check — if the action is high-risk, the system requires human approval before proceeding; if not, it moves straight to execution through the accessibility layer. From there, it checks whether the target UI element was identified — if yes, it interacts directly; if not, it uses the vision fallback. After that, the result is verified, memory and state are updated, a response is returned to the user, and the task is marked complete. The point of this slide is to show that safety checks — like clarification and the risk check — are built into the flow itself, not added as an afterthought."

Slide 14 — A.U.R.A. System Architecture

What to say:

"This is our full system architecture, organized into five layers, running on one device with no cloud dependency. At the top is the User Interaction Layer — the desktop interface and the Raspberry Pi IoT companion, which provides wake-word listening, a fingerprint sensor, and projector-based feedback. Below that is the Orchestration Layer — the Brain for intent interpretation, the Manager for planning, routing, and shared context, and the five specialized sub-agents, all sharing one task context. Next is the Intelligence Layer — lightweight reasoning through BitNet b1.58, complex reasoning through the quantized 7-to-8-billion model via llama.cpp, and a vision layer for UI fallback. Then the Execution Layer — accessibility APIs for direct UI targeting, and a restricted execution environment with controlled permissions and sandboxed operations. And finally, the Data Layer — memory in ChromaDB, state in SQLite, both protected with AES-256 encrypted local storage. As shown on the side, there are no outbound inference calls, and all data stays on the device — that's the air-gapped boundary we keep emphasizing."

Slide 15 — Security by Design

What to say:

"Security isn't a separate add-on in this project — it's built into the architecture, and this slide summarizes six principles. One, Local-First Processing — A.U.R.A. is designed to keep AI inference and user context on the local machine rather than relying on cloud AI APIs. Two, Zero-Trust Input Handling — external web content is treated as untrusted, which reduces the attack surface for indirect prompt injection. Three, Restricted Permissions — agents don't get unrestricted OS access; each operation is constrained to what that specific task actually needs. Four, Human-in-the-Loop — high-risk operations do not execute automatically; they wait for approval. Five, Encrypted Local Data — sensitive local state and memory are encrypted using AES-256. And six, Auditable Execution — important decisions and actions can be logged, so the user can review what A.U.R.A. attempted and why. On the right, you can see our two key pipelines: the untrusted content pipeline, where web content is sanitized and given a safe representation before the AI ever processes it, keeping instructions and data separate — and the high-risk action pipeline, where an action request goes through risk assessment, gets paused, and only proceeds after fingerprint verification and user approval. The key idea here is that physical presence is required — so even a compromised agent cannot approve a destructive action by itself."

Slide 16 — Project Conclusion

What to say:

"To bring it all together: A.U.R.A. stands for Air-gapped Unified Responsive Agent — a local AI system designed to turn natural-language commands into controlled computer actions. Four things summarize what we've built. One, Local Intelligence — AI reasoning can operate locally without needing cloud AI APIs. Two, Smart Automation — a multi-agent architecture lets complex tasks be broken down and handled by specialized agents. Three, Resilient Interaction — accessibility-based automation is supported by vision fallback when traditional UI information isn't available. And four, Privacy and Security — user data stays locally controlled through sanitization, restricted permissions, biometric approval, and encrypted storage. If we had to summarize the whole project in one sentence, it would be this: 'AI that can act on your computer — while the computer remains under your control.'"

Slide 17 — Thank You / Q&A

What to say:

"That brings us to the end of our presentation. Thank you for your time and attention. We'd now be happy to take any questions you have about A.U.R.A. — Air-gapped Unified Responsive Agent."

(When taking questions, use the "Ground Rules" at the top of this document — especially: don't overclaim, don't guess numbers, and be honest if something is still at the design stage rather than fully tested.)